

Pipe-and-Filter Architecture

Software Engineering Presentation

Presented by: Muhammad Zayan (2023554)

Muhammad Ahmad Amjad (2023361)

Abdullah Ejaz Janjua (2023038)

Instructor: Sir Israr Ahmad

What is **Pipe-and-Filter**?

 **Architectural style** for data-processing systems

 System is broken into small **independent steps**

 Steps are called **Filters**

 Connections between steps are **Pipes**

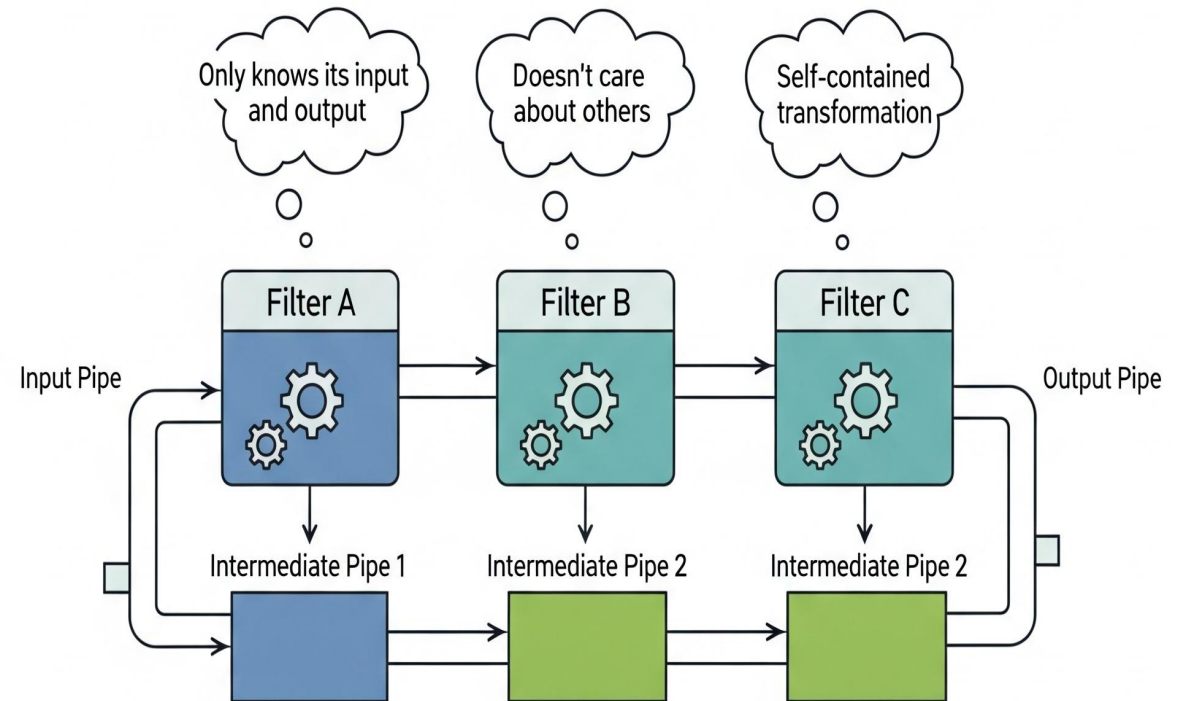
 **Output** of one filter becomes the **input** of the next

Simple Diagram

This diagram shows the basic flow.

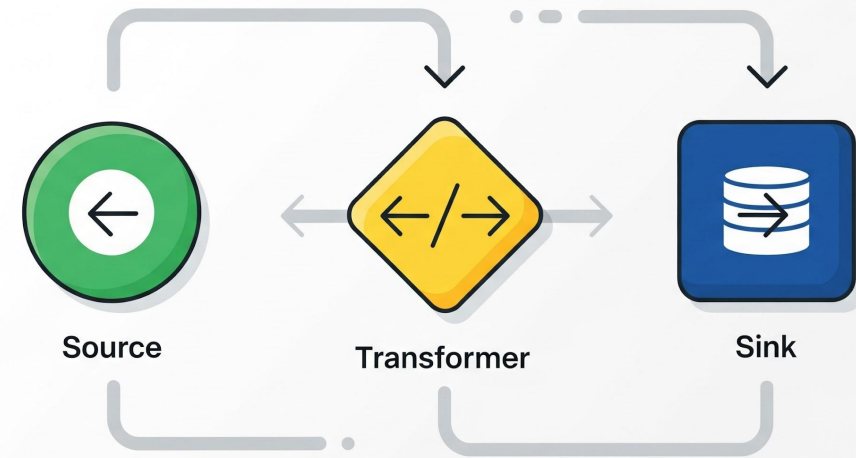
Each filter functions as a completely self-contained processing unit, independent of the rest of the pipeline. It doesn't need to understand how earlier or later filters work or what internal logic they use.

Its only job is to take data from an input pipe, apply its transformation, and pass the processed output to the next pipe for further handling.



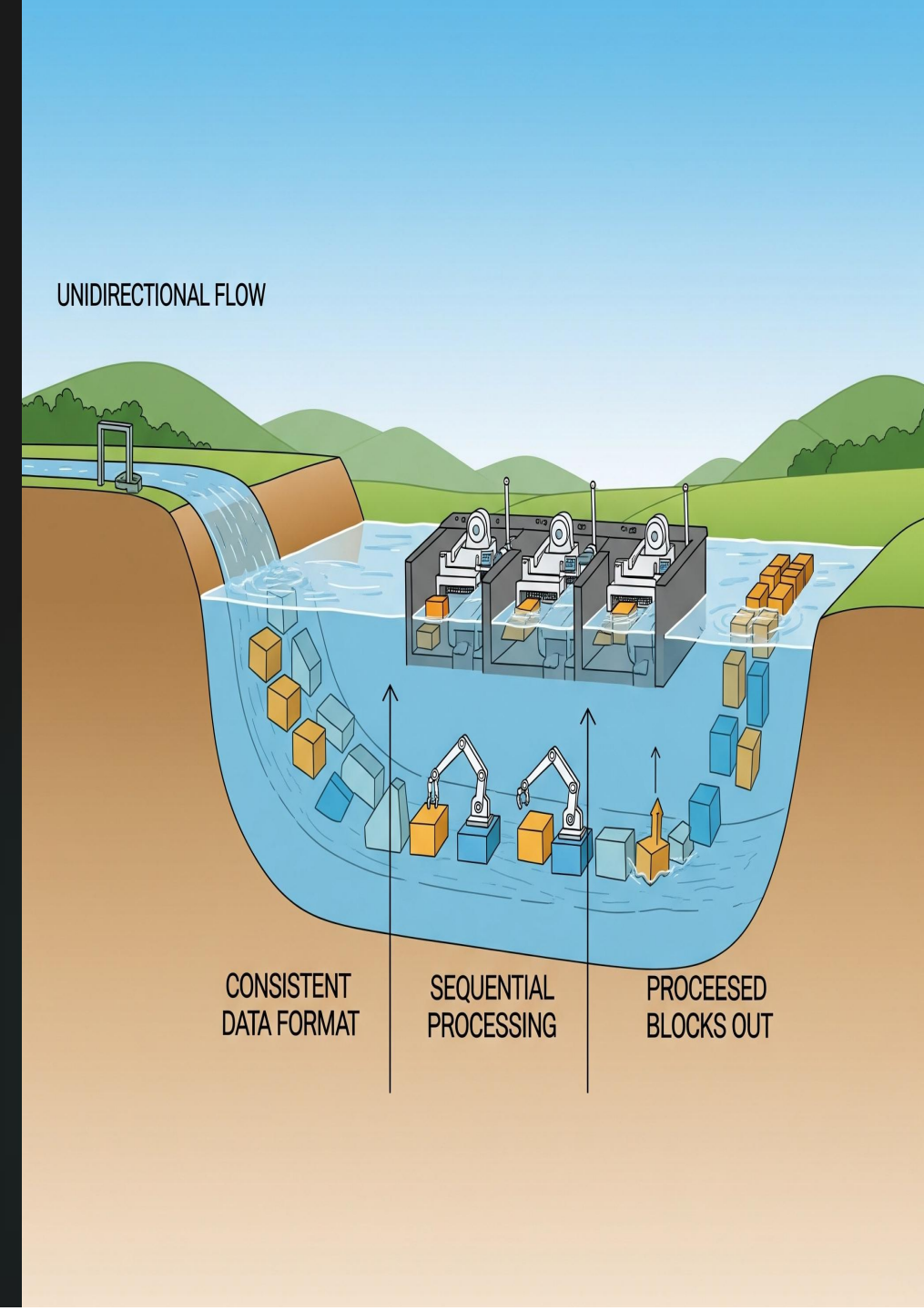
Types of Filters

- » Source: A filter with no input pipe; it initiates the data stream.
- » Sink: A filter with no output pipe; it terminates the data stream (e.g., saving to a database).
- » Transformer: A filter with both input and output, performing the core transformation.



Key Characteristics

- » Unidirectional Flow Data: moves in only one direction: from the output of Filter A to the input of Filter B. This one-way flow keeps the system simple, predictable, and easy to manage.
- » Consistent Data Format: All data moving through the pipes must follow a consistent, standardized format. This can be plain text (like Unix pipes) or structured data such as JSON. A consistent format ensures every filter can read and process the data correctly.
- » Sequential Processing: Filters operate in a clear, step-by-step order. The output of one filter becomes the input of the next, forming a smooth and organized pipeline.

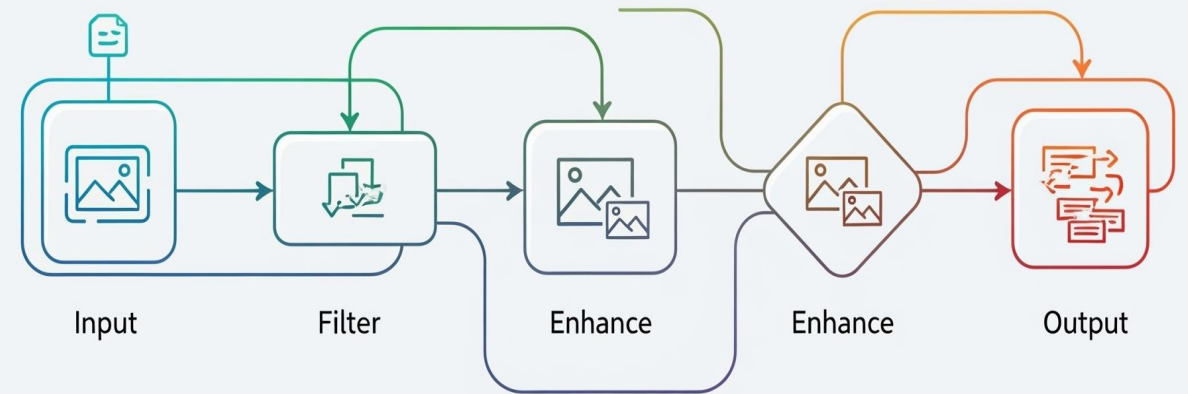


Non-Software Example

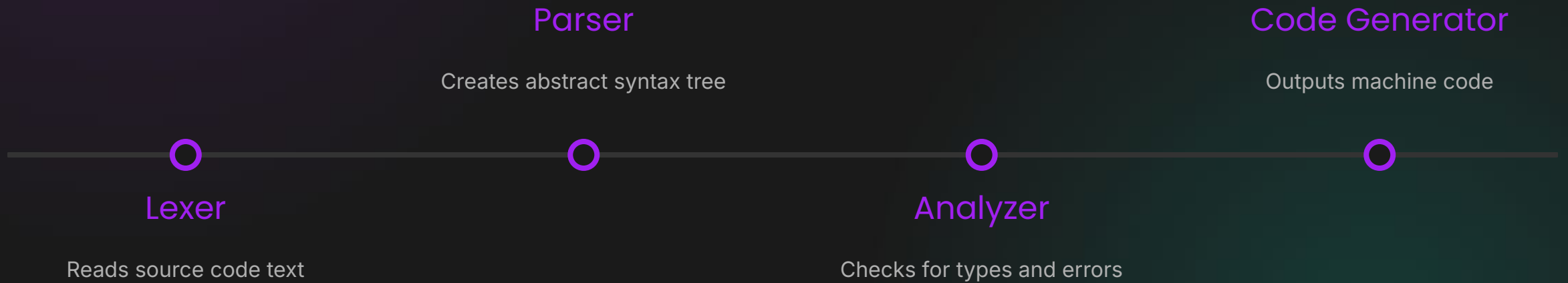
Image Processing Pipeline

This simple example shows how any linear flow of transformations is a natural fit for this architecture.

Resize → Filter → Adjust → Save



Example 1: Compilers

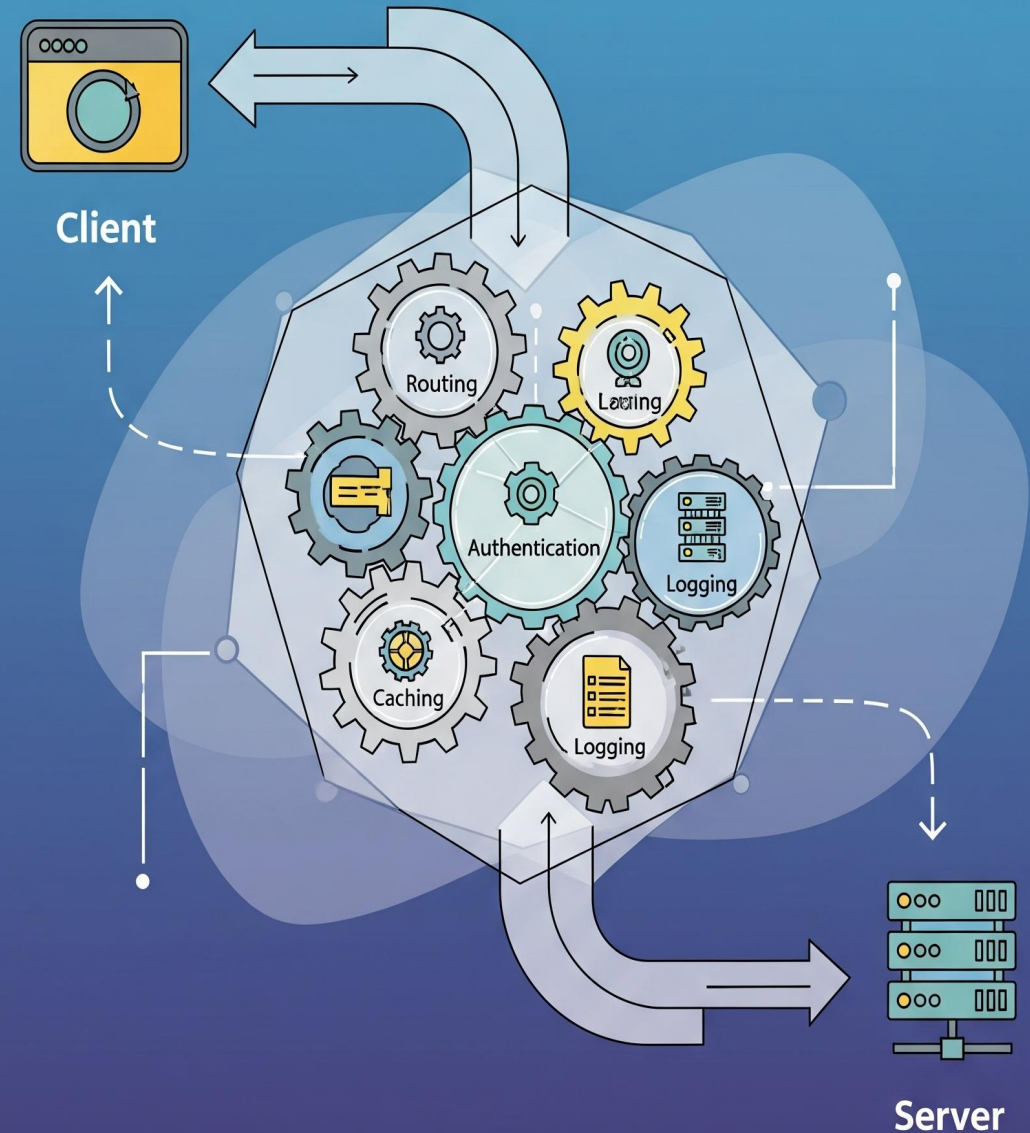


Example 2: Web Middleware

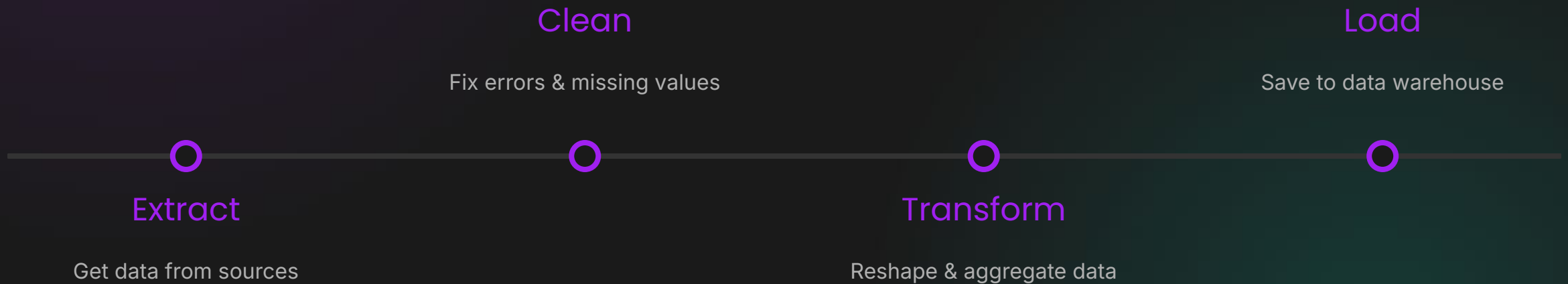
Used in Express.js, Django, ASP.NET

- » HTTP Request Arrives
- » Authentication Filter
- » Logging Filter
- » Validation Filter
- » Routing (Controller)

Web Middleware



Example 3: Data Processing (ETL)



Example 4: CI/CD Pipelines

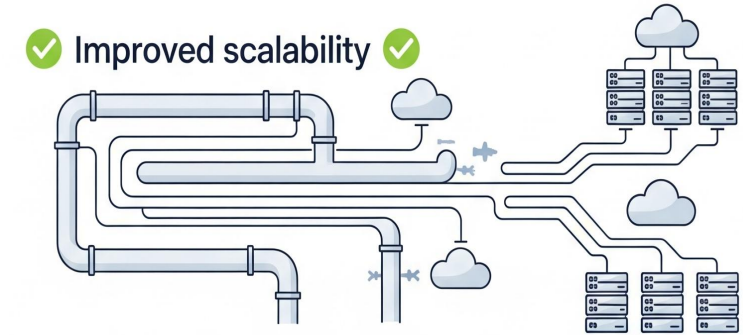
Every step is an independent filter.

- » Code Checkout
- » Build Project
- » Lint & Static Analysis
- » Run Unit Tests
- » Deploy to Production



Advantages

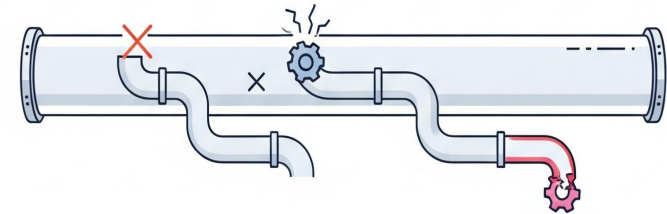
- ✓ High modularity & reusability
- ✓ Easy to extend, modify & maintain
- ✓ Parallel execution possible
- ✓ Good separation of concerns
- ✓ Good isolation i.e High fault tolerance



✓ Clear separation of concerns ✓



✓ Better fault isolation ✓



Disadvantages

- ❌ **Data overhead:** Passing data fully through multiple pipes is computationally expensive and generates contention for buffer and system resources.
- ❌ Requires uniform data format (e.g., text, JSON) & Filters agreed on format expect
- ❌ Not ideal for interactive systems
- ❌ Difficult when filters need shared state

When to Use / When Not to

When to Use

- ✓ Data-processing workflows
- ✓ Transformations in a clear sequence e.g Compilers, ETL pipelines
- ✓ Modularity are key

When NOT to Use

- ✗ Real-time interactive systems (e.g., GUIs)
- ✗ When components depend heavily on each other
- ✗ When state must be shared globally

Summary

- ★ **Filters are Independent Black Boxes:** Components are self-contained and decoupled, promoting high reusability and modularity.
- ★ **Requires Uniform Data Format:** A single, standardized data format is mandatory for all data transferred through the pipes.
- ★ **Unidirectional and Sequential Flow:** Information flows strictly in one direction, although this sequence can be managed to allow for parallel processing.
- ★ **Not Ideal for Interactive Systems:** Poorly suited for applications that demand immediate, two-way user interaction (e.g., GUIs).

Questions?

Thank you.